

Laboratorio 3 · Optimización

Ajusta, compara y defiende

Diseño de experimentos con metaheurísticas de trayectoria

Organización	Por parejas
Sesión de laboratorio	Lunes 5 de octubre, 2 horas (laboratorio 3 o 4)
Entrega 1 · Hoja de hipótesis	Martes 13 de octubre, 23:59 (Campus Virtual)
Entrega 2 · Práctica completa	Martes 13 de octubre, 23:59 (Campus Virtual)
Fichero	apellido1_apellido2_lab3.zip

1. Qué vais a hacer

En clase vimos seis algoritmos: búsqueda aleatoria, escalador de colina, recocido simulado, aceptación por umbral (TA), viaje de récord a récord (RRT) y Gran Diluvio. Salvo la búsqueda aleatoria, todos comparten el mismo esqueleto: generar un vecino, evaluarlo y decidir si se acepta. Lo único que cambia es la regla de aceptación.

Programar esas reglas es rápido. **Lo difícil, y lo que se evalúa en esta práctica, es lo que viene después:** cada algoritmo tiene sus “mandos” (la temperatura, el umbral, la desviación D, la lluvia...) y **nadie os va a decir qué valores poner**. Tenéis que averiguarlo vosotros con experimentos bien diseñados, y demostrarlo con datos.

La pregunta de la práctica: ¿cómo hay que ajustar cada algoritmo, y cuál recomendaríais para un problema que no conocéis? Responde la como lo haría un investigador: con hipótesis previas, un diseño justo, réplicas suficientes y conclusiones que se sostengan fuera de los datos con los que ajustasteis.

Al final entregaréis una configuración congelada de cada algoritmo. El profesorado la ejecutará en instancias del problema que no habéis visto (apartado 7). Una configuración que solo funciona en los casos de clase no pasará esa prueba.

2. Objetivos de aprendizaje

- Implementar las reglas de aceptación de seis metaheurísticas de trayectoria sobre un esqueleto común.
- Formular hipótesis comprobables sobre su comportamiento antes de ejecutarlas.
- Diseñar un experimento justo: factores, niveles, réplicas, presupuestos iguales y separación entre ajuste y evaluación.
- Analizar resultados estocásticos con estadística descriptiva y gráficos adecuados (y, opcionalmente, contrastes de hipótesis).

- Relacionar el valor de un parámetro con la estructura del problema, en lugar de tratarlo como un número mágico.
- Defender una recomendación con datos y reconocer las limitaciones del propio estudio.

3. Material

En el Campus Virtual tenéis una carpeta con estos ficheros:

Fichero	Qué es	¿Lo tocáis?
bloques.py	El problema de los bloques, el contador de evaluaciones, el esqueleto común y la función ejecutar().	No
algoritmos.py	La búsqueda aleatoria y las cinco reglas de aceptación, con huecos para completar.	Sí
configuracion.py	Vuestra configuración final: una función configurar(algoritmo, n, k, presupuesto).	Sí
verificar.py	Comprobaciones automáticas de la Fase 0, incluidas las trazas de clase.	No
ejemplo_uso.py	Cómo lanzar ejecuciones y guardar resultados en CSV. No es un experimento.	Como queráis

Solo hace falta Python 3 y la biblioteca estándar. Para los gráficos podéis usar matplotlib, pandas o lo que prefiráis. El Anexo A resume la interfaz de bloques.py.

4. El problema: bloques de k bits

Una solución es una cadena binaria x de longitud n , dividida en bloques disjuntos de k bits. Sea u el número de unos de un bloque. La aptitud es la suma sobre los bloques: $F(x) = \sum f(u_j)$, y se maximiza. En las cuatro funciones el óptimo global es la cadena de todo unos, con $F = n$.

En clase usamos siempre $k = 4$. El almacén admite cualquier k , con esta generalización:

Función	$f(u)$ para bloques de k bits	$k = 4$ ($u = 0 \dots 4$)	$k = 5$ ($u = 0 \dots 5$)
OneMax	u	0 1 2 3 4	0 1 2 3 4 5
Plateau	u si $u \geq k - 1$; 0 en otro caso	0 0 0 3 4	0 0 0 0 4 5
Royal Road	k si $u = k$; 0 en otro caso	0 0 0 0 4	0 0 0 0 0 5
Deceptive	$k - 1 - u$ si $u < k$; k si $u = k$	3 2 1 0 4	4 3 2 1 0 5

La vecindad es siempre la misma: **invertir un bit elegido al azar**. El presupuesto se mide en **evaluaciones de F**: cada llamada a problema.evaluar(x) gasta una y, al agotarlas, el almacén detiene el algoritmo. La ejecución también termina si se alcanza el óptimo.

5. Reglas del juego

Estas reglas convierten «probar cosas» en un experimento. Saltarse cualquiera de ellas invalida la parte afectada de la memoria.

#	Regla	Por qué
R1	Todos los algoritmos se comparan con el mismo presupuesto de evaluaciones en la misma instancia.	Si uno tiene más evaluaciones, la comparación no dice nada del algoritmo.
R2	Cada algoritmo dispone del mismo presupuesto de ajuste: el mismo número de configuraciones probadas. Declaradlo en el diseño.	Si ajustáis mucho uno y poco otro, gana el que más habéis ajustado, no el mejor.
R3	Ninguna afirmación se apoya en una sola ejecución. Mínimo 30 réplicas en la evaluación final; en el ajuste, las que justifiéis.	Los algoritmos usan azar: una ejecución puede salir bien por suerte.
R4	Las semillas de ajuste y las de evaluación final son distintas, y las de evaluación se fijan antes de ejecutar.	Si evaluáis con los mismos datos con los que ajustasteis, el resultado sale inflado.
R5	Todo resultado de la memoria se puede reproducir con vuestro código y las semillas indicadas.	Es la única forma de que otro pueda comprobarlo.
R6	Usad siempre el generador de azar que recibís: <code>self.rng</code> o <code>rng</code> .	Si no, la semilla no controla el azar y R5 falla.

Nota sobre R6: dentro de las reglas, el azar se pide a `self.rng` (en la búsqueda aleatoria, al parámetro `rng`), nunca al módulo `random` directamente. `verificar.py` lo comprueba.

6. Fases

Fase 0 · Implementación y verificación (en la sesión, ≈ 1 hora)

Completad en `algoritmos.py` la búsqueda aleatoria y las cinco reglas de aceptación. Los nombres de los parámetros obligatorios están en la cabecera del fichero; podéis añadir los que queráis (por ejemplo, el esquema de enfriamiento o el ritmo de bajada del umbral), siempre con un valor por defecto.

Ejecutad `python verificar.py` hasta que todo salga OK. Entre otras cosas, reproduce las trazas de las diapositivas: desde 0000 0000, el TA con $U = 3$, el RRT con récord 6 y $D = 3$ y el Gran Diluvio con nivel 2,5 aceptan la secuencia $6 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 7$ hasta 1111 0000.

Fase 1 · Hipótesis (en la sesión; Entrega 1)

Antes de lanzar ningún experimento, escribid vuestras predicciones en la hoja del Anexo B. **Mínimo ocho:** una por algoritmo y dos transversales (comparaciones entre algoritmos o entre funciones). Cada hipótesis tiene tres partes:

- **Predicción:** qué esperáis que ocurra, de forma que se pueda refutar.
- **Razonamiento:** por qué, a partir de lo visto en clase y de la forma de las funciones.
- **Cómo la comprobaréis:** qué vais a medir y qué resultado la confirmaría o la refutaría.

Ejemplo (no lo copiéis): «Predicción: la búsqueda aleatoria no alcanzará el óptimo de OneMax con $n = 32$ en ninguna de 30 ejecuciones de 20 000 evaluaciones. Razonamiento: la probabilidad de acertar en un intento es $1/2^{32}$, y con 20 000 intentos la probabilidad de acertar alguna vez es despreciable. Comprobación: tasa de éxito en 30 réplicas; se refuta si es mayor que 0.»

La hoja se entrega al terminar la sesión y **no se modifica después**. En la memoria diréis cuáles se confirmaron y cuáles no. Una hipótesis refutada y bien explicada vale tanto como una confirmada; una hipótesis escrita después de ver los datos no vale nada.

Fase 2 · Diseño del experimento

Para cada algoritmo, rellenad una tabla de diseño (plantilla en el Anexo C) que responda a:

- **Factores:** qué parámetros vais a variar. ¿Hay alguno que no sea un número (un esquema de enfriamiento, aceptar o no los vecinos iguales)?
- **Niveles:** qué valores de cada factor y por qué esos. ¿Escala lineal o logarítmica?
- **Factores fijos:** qué dejáis constante (función, n , k , presupuesto) y con qué valor.
- **Respuesta:** qué medís. Como mínimo, el mejor F alcanzado y si se llega al óptimo; recomendable, las evaluaciones hasta el óptimo.
- **Réplicas y semillas** de ajuste.
- **Presupuesto de ajuste:** cuántas configuraciones en total, igual para todos los algoritmos (R2).

Pensad también en las interacciones: ¿el mejor valor de un parámetro depende del valor de otro, o de la función, o de n ?

Fase 3 · Ajuste

Ejecutad el diseño sobre vuestras instancias de ajuste. Guardad todas las ejecuciones en CSV (una fila por ejecución) y no solo las medias: las necesitaréis para los gráficos. Mirad la dispersión, no solo el promedio: dos configuraciones con la misma media pueden comportarse de forma muy distinta.

Fase 4 · De números a reglas: configuracion.py

Vuestra configuración final no es un número por algoritmo, sino una **función de la instancia**:

```
def configurar(algoritmo, n, k, presupuesto):  
    ...  
    return {"D": ...}      # por ejemplo, para el RRT
```

Recibe la longitud n , el tamaño de bloque k y el presupuesto, pero **no qué función f se va a usar**. Por tanto debe funcionar razonablemente en las cuatro. Si un parámetro depende de n , de k o del presupuesto, la memoria debe explicar por qué con vuestros datos; por ejemplo, mostrando cómo cambia el mejor valor al variar k o n . Comprobad vuestra regla en instancias distintas de las de ajuste antes de congelarla.

Fase 5 · Evaluación final y comparación

Con la configuración congelada, ejecutad los seis algoritmos sobre las cuatro funciones en **al menos dos tamaños**, con 30 réplicas y semillas de evaluación (R4). Para cada combinación, informad de:

- mediana y rango intercuartílico del mejor F (o media y desviación típica, justificando la elección);
- tasa de éxito: proporción de réplicas que alcanzan el óptimo;
- evaluaciones hasta el óptimo, solo en las réplicas con éxito.

Presentadlo con diagramas de caja o gráficos equivalentes y una tabla resumen. **Opcional (suma en «Análisis»):** contrastes de Wilcoxon (Mann-Whitney) entre pares de algoritmos, con su nivel de significación y una corrección por comparaciones múltiples.

Fase 6 · Conclusiones

La memoria debe responder, con vuestros datos, a estas preguntas:

1. ¿Qué hipótesis de la Fase 1 se confirmaron y cuáles no? En las refutadas, ¿qué fallaba en vuestro razonamiento?
2. Para cada algoritmo, ¿de qué depende su parámetro clave? ¿Cómo lo habéis traducido en una regla dentro de configurar()?

3. ¿Hay un algoritmo que gane en todas las funciones? Si no, ¿por qué no?
4. Si os dieran un problema nuevo del que no sabéis nada, ¿qué algoritmo y qué configuración usaríais, y por qué?
5. ¿Cuáles son las limitaciones de vuestro estudio? ¿Qué no habéis probado y podría cambiar las conclusiones?

7. La prueba final: instancias ocultas

Tras la entrega, el profesorado ejecutará vuestros `algoritmos.py` y `configuracion.py` con el `bloques.py` oficial sobre un conjunto de **instancias que no conocéis**. Pueden cambiar la función, n , k y el presupuesto. Se harán 30 réplicas con semillas propias.

Para cada algoritmo e instancia se calcula:

$$\text{puntos} = 0,5 \cdot \text{tasa de éxito} + 0,5 \cdot (\text{media del mejor F}) / \text{óptimo}$$

La puntuación del grupo es la media sobre instancias y sobre los cinco algoritmos de trayectoria; la búsqueda aleatoria se informa como línea base pero no puntúa. Se publicará una clasificación.

Por qué existe esta prueba. En clase visteis valores concretos de U , D y otros parámetros que funcionaban en los ejemplos de $k = 4$. Copiarlos sin entender por qué funcionaban no basta: si la instancia cambia, el valor correcto también puede cambiar. Lo que se premia es haber entendido de qué depende cada parámetro.

Una ejecución que lance una excepción, supere el presupuesto o devuelva un mejor F que no coincide con $F(\text{mejor_x})$ puntúa 0. Probad vuestro `configurar()` con valores de n , k y presupuesto distintos de los de clase.

8. Entregables

Entrega 1 (lunes 5, al terminar la sesión): la hoja de hipótesis (Anexo B) en PDF. Entrega 2 (martes 13, 23:59): un único `apellido1_apellido2_lab3.zip` con:

Contenido	Detalle
<code>algoritmos.py</code>	Las seis implementaciones; verificar.py todo OK.
<code>configuracion.py</code>	Vuestra configuración congelada (apartado 7).
<code>experimentos/</code>	Los scripts que generan todos los resultados, con un README de una página que diga cómo reproducirlos.
<code>resultados/</code>	Los CSV con todas las ejecuciones (ajuste y evaluación).
<code>memoria.pdf</code>	Máximo 10 páginas (sin contar anexos). Estructura en el apartado 8.1.

8.1. Estructura de la memoria

1. Hipótesis (copia literal de la Entrega 1).
2. Diseño: tablas del Anexo C, presupuestos y semillas.
3. Resultados del ajuste: gráficos que justifiquen cada elección de parámetros.
4. `configurar()`: la regla de cada algoritmo y por qué.
5. Evaluación final: tabla resumen y diagramas de caja.
6. Conclusiones: respuestas a las cinco preguntas de la Fase 6.

La práctica se defenderá oralmente, en unos 5 minutos, según el calendario de defensas.

9. Evaluación

Criterio	Qué se valora	Peso
Implementación	Las seis reglas son correctas, verificar.py da todo OK y el código es legible.	10 %
Hipótesis y diseño	Hipótesis refutables y razonadas; factores, niveles, réplicas y presupuestos justificados.	25 %
Rigor experimental	Cumplimiento de R1–R6; resultados reproducibles.	20 %
Análisis	Estadística descriptiva adecuada, gráficos claros, atención a la dispersión. Wilcoxon suma aquí.	20 %
Conclusiones y defensa	Respuestas apoyadas en los datos, hipótesis contrastadas, limitaciones reconocidas.	15 %
Instancias ocultas	Puntuación de vuestra configuración en la prueba del apartado 7.	10 %

Limitaciones de nota:

- Una conclusión apoyada en una sola ejecución no puntúa.
- Comparar algoritmos con presupuestos de evaluación distintos (R1) anula el criterio de rigor.
- Si los resultados de la memoria no se reproducen con vuestro código y semillas, se califican los que se reproducen.
- bloques.py no se entrega: cualquier cambio en él no tiene efecto.

Anexo A · Interfaz de bloques.py

Elemento	Descripción
Problema(funcion, n, k=4, presupuesto=20000)	Crea una instancia. Usad uno nuevo por ejecución.
problema.evaluar(x)	Devuelve F(x) y gasta una evaluación. Lanza PresupuestoAgotado al agotarlas.
problema.aleatoria(rng)	Cadena aleatoria de n bits (no gasta evaluaciones).
problema.vecino(x, rng)	Copia de x con un bit invertido (no gasta evaluaciones).
problema.optimo / .evaluaciones / .quedan()	Óptimo (= n), evaluaciones gastadas y restantes.
Resultado	mejor_x, mejor_f, evals_hasta_optimo (None si no llegó), historia [(evaluación, mejor F)].
ejecutar(algoritmo, parametros, problema, semilla)	Una ejecución completa; devuelve un Resultado. algoritmo ∈ {aleatoria, escalador, recocido, umbral, rrt, diluvio}.

Esqueleto que ejecuta bloques.py para los cinco algoritmos de trayectoria:

```
x = aleatoria(); fx = evaluar(x)
mientras no se agote el presupuesto y no se alcance el óptimo:
    y = vecino(x); fy = evaluar(y)
    si regla.aceptar(fx, fy, mejor_f): x, fx = y, fy
    actualizar mejor_f
    regla.tras_iteracion(aceptado)
```

Anexo B · Hoja de hipótesis (Entrega 1)

Grupo: _____ Integrantes: _____ /

#	Predicción	Razonamiento	Cómo se comprueba
H1 · Aleatoria			
H2 · Escalador			
H3 · Recocido			
H4 · Umbral			
H5 · RRT			

#	Predicción	Razonamiento	Cómo se comprueba
H6 · Diluvio			
H7 · Transversal			
H8 · Transversal			

Las filas se pueden ampliar. Una hipótesis por fila.

Anexo C · Plantilla de tabla de diseño

Una por algoritmo. El ejemplo muestra cómo rellenarla, no qué valores elegir.

Campo	Ejemplo: escalador de colina
Pregunta que responde	¿Influye aceptar los vecinos de igual valor? ¿Depende de la función?
Factores y niveles	aceptar_iguales $\in$ {True, False}
Factores fijos	$n = 32$, $k = 4$, presupuesto = 20 000; las cuatro funciones
Respuesta	Mejor F; éxito (sí/no); evaluaciones hasta el óptimo
Réplicas y semillas (ajuste)	20 réplicas, semillas 1–20
Configuraciones probadas	2 de un presupuesto de ajuste de ___ por algoritmo
Resultado esperado ($\rightarrow$ Hipótesis)	H2

Anexo D · Errores frecuentes

- Reutilizar el mismo objeto Problema en varias ejecuciones: el contador sigue sumando y la segunda ejecución no tiene presupuesto.
- Usar `random.random()` en lugar de `self.rng.random()`: las ejecuciones dejan de ser reproducibles.
- En el recocido, equivocar el signo de Δ al maximizar: la probabilidad de aceptar un vecino peor debe estar entre 0 y 1.
- En el RRT, comparar con la solución actual en lugar de con el récord: entonces ya no es RRT, es TA con umbral fijo.
- Hacer depender el ritmo de bajada del umbral o de la temperatura de un número fijo de iteraciones en lugar del presupuesto: al cambiar el presupuesto, el esquema se rompe.
- Ajustar y evaluar con las mismas semillas (R4).
- Elegir la configuración por la media sin mirar la dispersión ni la tasa de éxito.
- Copiar en `configurar()` los valores de las diapositivas sin comprobar si siguen funcionando al cambiar n o k .